

Turkish Dependency Parsing with Morphological Feature Injection and Treebank-Specific Adapters

İpek Akkuş[†] and Tarık Emre Tıraş[‡]

Boğaziçi University

34342 Bebek, Istanbul, Turkey

[†]Department of Computer Engineering

[‡]Department of Linguistics

{ipek.akkus, tarik.tiras}@bogazici.edu.tr

Abstract

We present a Turkish dependency parsing system built on the SuPar second-order TreeCRF parser (Zhang et al., 2020) with BERTurk (Schweter, 2020), evaluated on three Universal Dependencies treebanks (BOUN, IMST, PUD). Our best single system, A2b (BiLSTM + BERTurk + POS features), achieves **74.73 LAS** on BOUN, outperforming all off-the-shelf parsers and approaching the published biaffine baseline of Türk et al. (2022). A STEPS-style ablation (Grünwald et al., 2021) shows that adding a BiLSTM yields +10.88 LAS concentrated in label accuracy rather than head selection, revealing that sequential context is critical for reading Turkish agglutinative suffixes. Treebank-specific Houlsby adapters (Houlsby et al., 2019; Üstün et al., 2022) achieve the strongest zero-shot transfer to PUD (63.87 LAS). Morphological feature injection from the UD FEATS column (Özateş et al., 2022) adds +9.83 LAS over the baseline on BOUN, with gains driven by case, number, and tense features disambiguating Turkish dependency labels. McNemar’s significance tests confirm that all improvements over the baseline are statistically significant ($p < 0.001$).

1 Introduction

Dependency parsing assigns each token a syntactic head and a labeled dependency relation, producing a directed tree over the sentence. Neural parsers achieve strong results on resource-rich languages such as English and Chinese, but Turkish presents well-documented challenges. Turkish is an agglutinative language (Nivre et al., 2016): words encode tense, person, number, case, and possession as stacked suffixes on a root, producing a large and sparse surface vocabulary. Word order is also relatively free, and cross-lingual transfer is difficult because Turkish morphology and syntax differ substantially from typical high-resource source languages (Zhou and Kübler, 2025).

Turkish has three Universal Dependencies treebanks, BOUN, IMST, and PUD, that differ in size, domain, and annotation conventions. A parser trained on one treebank does not transfer cleanly to the others, and naive concatenation introduces annotation inconsistencies that degrade label accuracy. These properties make Turkish a demanding test case for both encoder choice and multi-treebank training strategies.

This paper presents a Turkish dependency parsing system grounded in six course papers. We build a strong baseline using SuPar’s second-order TreeCRF parser (Zhang et al., 2020) with BERTurk (Schweter, 2020) and extend it in three directions: a STEPS-style architectural ablation (Grünwald et al., 2021) to identify which components matter for Turkish; treebank-specific Houlsby adapters (Houlsby et al., 2019; Üstün et al., 2022) for joint multi-treebank training without annotation interference; and explicit morphological feature injection from the UD FEATS column and Zemberek-NLP (Özateş et al., 2022; Akın and Akın, 2007) to provide the label classifier with direct morphological signal. Section 2 reviews related work, Section 3 describes each component, Section 4 details the experimental setup, and Section 5 presents results and analysis.

2 Background and Related Work

Neural dependency parsing. Graph-based biaffine parsers are the standard for UD parsing. Zhang et al. (2020) built upon the biaffine parser (Dozat and Manning, 2017) with a second-order TreeCRF objective. The architecture combines a pre-trained transformer encoder with an optional BiLSTM, biaffine arc and label scoring heads, and a global TreeCRF objective that enforces valid tree structure via an inside algorithm. We use SuPar’s CRF2oDependencyParser directly for our baseline model.

Architectural ablation. Grünewald et al. (2021) tested which parts of a parser are necessary. Their STEPS framework isolates the effect of three components, which are the BiLSTM layer, a POS auxiliary objective, and the encoder, through binary comparisons by removing one component at a time. It measures the change in LAS. We ran these same tests on Turkish BOUN data.

Adapters for multi-treebank training. Hounsby et al. (2019) introduced adapter modules. These are small blocks added after each transformer sublayer. The main encoder stays frozen. Üstün et al. (2022) introduce UAdapter, which inserts Hounsby adapter modules conditioned on URIEL typological feature vectors into mBERT for zero-shot cross-lingual dependency parsing. We adapt the adapter architecture to the multi-treebank setting within a single language: treebank identity replaces typological conditioning, and one adapter set per treebank absorbs annotation-style differences between BOUN and IMST while a shared BERTurk encoder and biaffine head capture general Turkish syntax.

Turkish dependency parsing. Türk et al. (2022) introduced the BOUN treebank. They reported a BERTurk baseline of 75.50 LAS. Özateş et al. (2022) present a hybrid deep dependency parser for Turkish that injects morphological features like case, number, person, aspect, voice, and polarity, alongside a neural backbone, reporting strong results on BOUN and IMST. Results of this hybrid system reached 82.10 LAS on BOUN. Their finding that explicit morphological signals improve label prediction directly motivates our Extension.

Cross-lingual context. Zhou and Kübler (2025) studied cross-lingual dependency parsing for languages with unique scripts, demonstrating the limitations of annotation projection and naive multilingual transfer across typologically distant languages. Their findings reinforce the case for language-specific pretraining and treebank-adapted architectures, motivating our choice of BERTurk over XLM-R and our use of treebank-specific adapters rather than a generic multilingual model.

3 System Description

All four components use SuPar’s CRF2oDependencyParser (Zhang et al., 2020) with BERTurk (dbmdz/bert-base-turkish-uncased) as the shared backbone. Implementation details and notebook links are provided in the Appendix.

3.1 Baseline

We train one model per treebank on top of BERTurk using SuPar’s CRF2oDependencyParser (Zhang et al., 2020). The hidden size is 800, dropout is 0.33, and the batch size is 5,000 tokens. We use AdamW optimization with early stopping. We tested different learning rates and decided to use 1e-3 for BOUN and 2e-3 for IMST. A shared rate of 2e-3 caused the model to fail on BOUN, and 5e-4 was too low for both. The per-treebank split rates fixed this issue. The BERT learning rate is 2e-5. PUD has no training split and is used only as a test set. Two compatibility patches are required: Transformers is pinned to 4.35.2, and torch.load is patched for PyTorch 2.6.

3.2 Architectural Ablation

Following Grünewald et al. (2021), we run the STEPS protocol on BOUN with three binary comparisons, each changing exactly one component while holding all others fixed. Similar variants share a checkpoint (A1a = A3a = baseline; A2a = A1b). All ablation variants use 50 epochs / patience 15.

- **A1 - BiLSTM layer:** A1a (BERT encoder only) vs A1b (BiLSTM on top of BERT). A1a reuses the baseline checkpoint directly. A1b adds a BiLSTM layer on top of BERTurk (encoder=lstmlstm, BERTurk features as LSTM input) with Adam optimizer and ExponentialLR decay, LR 2×10^{-3} , momentum parameters (0.9, 0.9).

- **A2 — POS input features:** *A2a (LSTM + BERT, no POS)* vs *A2b (LSTM + BERT + POS)*. A2a reuses A1b. A2b adds Universal POS tags as additional input features to the BiLSTM via SuPar’s `feat=[tag]` mechanism. This feeds POS as LSTM input rather than as an auxiliary prediction objective; we describe it as POS input features throughout.
- **A3 — Encoder choice:** *A3a (BERTurk)* vs *A3b (XLM-R-base)*. A3a reuses `baseline_boun.pt`. A3b trains with XLM-R-base (`xlm-roberta-base`) using a lower task LR of 5×10^{-4} and encoder LR of 5×10^{-6} (4× lower than BERTurk), with batch size 2,000 (reduced for VRAM), to accommodate XLM-R’s larger vocabulary and sparser Turkish subword coverage.

3.3 Multi-Treebank Adapters

We implement Houlisby adapters from scratch in PyTorch, as the `adapter-transformers` library is incompatible with `transformers==4.35.2` required by SuPar. Each adapter follows the UDapter configuration Üstün et al. (2022): `LayerNorm` $\rightarrow$ `Linear(H $\rightarrow$ 64)` $\rightarrow$ `GELU` $\rightarrow$ `Linear(64 $\rightarrow$ H)` + residual, with near-zero initialization. Two adapters are inserted per BERTurk layer (after self-attention and after the feed-forward sublayer), yielding approximately 2.4M trainable parameters per treebank against 110.6M frozen BERTurk parameters. A separate adapter set is maintained for each treebank (BOUN and IMST; PUD is test-only).

The BERTurk encoder is fully frozen; only adapter weights and the shared biaffine head (~ 5.9 M parameters) are updated. Training uses a round-robin schedule alternating BOUN and IMST batches over 30 epochs (patience 10), with per-treebank AdamW optimizers (LR 1×10^{-3} , linear warmup over 10% of steps). Best adapter weights are selected by dev LAS per treebank.

3.4 Morphological Feature Injection

We compare three variants on BOUN. V1 is the baseline (BERTurk + SuPar, no morphological features). We inject morphological features by mapping each `Key=Value` pair from the CoNLL-U FEATS column to a 32-dimensional learned embedding. A token’s feature embeddings are summed into one vector, concatenated with BERTurk’s first-subword output, and projected back to the parser’s hidden size via a linear layer initialized near-identity. The feature vocabulary contains 85 unique `Key=Value` pairs (87 with PAD and UNK) built over all three treebanks.

- **V2:** BERTurk + UD FEATS embeddings as described above.
- **V3:** V2 plus a second 32-dimensional embedding table built from Zemberek-NLP (Akın and Akın, 2007) morpheme labels (86 labels + PAD/UNK), concatenated with the UD FEATS vector before projection.

The injection point is after BERTurk’s `encode()` and before the biaffine MLP layers; SuPar’s model forward is monkey-patched to accept an additional morphological index tensor. All variants initialize from the Phase 2 BOUN baseline checkpoint. V2 uses batch 2,000 (reduced to fit BERTurk + morph modules in GPU memory), 10 epochs, patience 10, LR 1×10^{-3} , BERT LR 2×10^{-5} . V3 uses batch 5,000, 10 epochs, patience 5, LR 2×10^{-3} .

4 Experimental Setup

Data. We use three UD Turkish treebanks. **UD Turkish-BOUN** Türk et al. (2022) is the primary training corpus: 7,803 / 979 / 979 sentences for train / dev / test, drawn from the Turkish National Corpus. **UD Turkish-IMST** covers news and Wikipedia: 3,435 / 1,100 / 1,100 sentences. **UD Turkish-PUD** (1,000 test-only) serves as a cross-domain zero-shot evaluation set across all phases. Gold segmentation from the CoNLL-U files is used throughout; no automatic tokenizer is applied.

Evaluation. We calculate UAS and LAS using SuPar’s `parser.evaluate()`. All systems are evaluated with gold tokenization, making our numbers directly comparable with published results. We also used McNemar’s test (McNemar, 1947) for significance tests.

Off-the-shelf baselines. We evaluate Stanza (Qi et al., 2020) and UDPipe (Straka and Straková, 2017) on the same test splits for comparison.

Hardware. We used Google Colab Pro with Nvidia T4 GPUs for all runs. The XLM-R rerun used an A100 instance.

5 Results and Discussion

5.1 Main Results

Table 1 reports results across all systems on BOUN, IMST, and zero-shot PUD alongside published and off-the-shelf comparison systems. Our BOUN baseline achieves LAS 61.96 and our IMST baseline achieves LAS 62.61, both below the published biaffine baseline of Türk et al. (2022) and the hybrid system of Özateş et al. (2022), which uses hand-written rules and richer morphological features. Stanza reaches 60.64 LAS on BOUN and UDPipe 2 reaches 63.31; our baseline is competitive with both, confirming that our training setup reproduces the off-the-shelf benchmark. These results serve as lower bounds for subsequent phases.

The A2b variant (BiLSTM + BERTurk + POS) is the strongest single system, achieving LAS 74.73 on BOUN, close to Türk et al. (2022) and outperforming all off-the-shelf parsers. The Houlsby adapter achieves the highest zero-shot LAS on PUD (63.87).

System	BOUN		IMST		PUD (zero-shot)	
	UAS	LAS	UAS	LAS	UAS	LAS
<i>Published baselines</i>						
Türk et al. (2022) — biaffine	—	75.50	—	—	—	—
Özateş et al. (2022) — hybrid+morph	88.20	82.10	88.90	83.40	—	—
<i>Off-the-shelf</i>						
Stanza (BOUN / IMST model)	78.78	60.64	78.42	63.15	—	—
UDPipe 2 (BOUN / IMST model)	80.48	63.31	83.94	65.29	—	—
<i>Our systems — baseline</i>						
Baseline — BERTurk + SuPar	77.26	61.96	76.32	62.61	76.26	60.41
<i>Our systems — ablation (§3.2)</i>						
A1b — BiLSTM + BERTurk	82.06	72.84	—	—	—	—
A2b — BiLSTM + BERTurk + POS *	82.62	74.73	—	—	83.19	69.41
A3b — XLM-R (corrected schedule)	76.05	49.39	—	—	74.52	48.90
<i>Our systems — adapters (§3.3)</i>						
Adapter (BOUN)	77.96	68.74	—	—	76.23	63.87
Adapter (IMST)	—	—	80.65	70.08	74.09	62.72
<i>Our systems — morphological injection (§3.4)</i>						
Morph V2 — BERTurk + UD FEATS	79.92	71.79	—	—	76.43	64.05
Morph V3 — V2 + Zemberek	80.15	71.79	—	—	76.23	63.65

Table 1: Full results on BOUN, IMST, and zero-shot PUD. * = best single system. Section references correspond to system description subsections. Ablation variants are trained on BOUN only.

5.2 Ablation Results

Table 2 reports the STEPS ablation results on BOUN with McNemar’s significance tests. All three comparisons are statistically significant at $p < 0.01$.

Comp.	Pair	UAS	LAS	Δ LAS	b	c	p -value
A1 — BiLSTM	A1a: BERTurk, no BiLSTM	77.26	61.96	+10.88	15	102	<0.0001 ***
	A1b: BiLSTM + BERTurk	82.06	72.84				
A2 — POS	A2a: BiLSTM + BERTurk, no POS	82.06	72.84	+1.89	27	51	0.0092 **
	A2b: BiLSTM + BERTurk + POS	82.62	74.73				
A3 — Encoder	A3a: BERTurk	77.26	61.96	−12.57	—	—	—
	A3b: XLM-R	76.05	49.39				

Table 2: STEPS-style ablation on BOUN test. McNemar’s continuity-corrected χ^2 test. b : sentences where the first system is fully correct and the second is not; c : the reverse.

A1 — BiLSTM. Adding a BiLSTM on top of BERTurk yields +10.88 LAS ($p < 0.0001$). The LAS gain is much larger than the UAS gain (+4.80), indicating that sequential context primarily helps the label classifier read Turkish morphological suffixes rather than improving head identification.

A2 — POS feature. Adding POS-tag features contributes a further +1.89 LAS ($p = 0.0092$). The effect is small but significant; POS tags provide explicit morphosyntactic distinctions that help predict dependency labels in Turkish (Özateş et al., 2022).

A3 — BERTurk versus XLM-R. BERTurk outperforms XLM-R by 12.57 LAS even after retraining A3b with a corrected lower learning rate. A Turkish-pretrained encoder provides cleaner morpheme boundaries and better subword coverage for Turkish parsing (Zhou and Kübler, 2025).

5.3 Adapter Results

The BOUN adapter reaches LAS 68.74 on BOUN (+6.78 over baseline; Table 1) and the IMST adapter reaches LAS 70.08 on IMST (+7.47 over baseline). Despite these gains, A2b (74.73) remains substantially stronger on BOUN. On PUD, the BOUN adapter achieves 63.87 LAS, which is the highest zero-shot LAS across all systems, suggesting that the frozen BERTurk encoder retains broad Turkish syntactic knowledge. The McNemar test confirms the adapter gain over the baseline is significant on both BOUN ($p < 0.0001$) and PUD ($p = 0.0095$). However, A2b significantly outperforms the adapter on BOUN ($p < 0.0001$, $b = 100$, $c = 34$), and the adapter is significantly outperformed by Morph V2 on BOUN ($p < 0.0001$, $b = 178$, $c = 7$).

5.4 Morphological Feature Injection

V2 (BERTurk + UD FEATS) reaches LAS 71.79 on BOUN and 64.05 on PUD (Table 1), representing gains of +9.83 and +3.64 respectively over the baseline. The improvement is highly significant on both BOUN ($p < 0.0001$) and PUD ($p < 0.0001$). The most frequent FEATS keys, listed as Number, Person, Case, Polarity, and Tense, are the primary cues for UD label assignment in Turkish, distinguishing for example `nsubj` from `obj` via case marking and `nmod` from `obl` via postpositional case. V3 (V2 + Zemberek) achieves the same LAS 71.79 on BOUN. Although V2 vs V3 is significant on BOUN ($p = 0.0003$), this reflects minor prediction differences rather than a meaningful accuracy gap; on PUD the difference is not significant ($p = 0.2482$). The UD FEATS column is therefore sufficient as a morphological signal for UD-based parsing.

5.5 McNemar’s Significance Tests

Table 3 reports significance tests across all system pairs on BOUN and PUD zero-shot.

Comparison	b	c	χ^2	p -value
<i>BOUN test</i>				
Baseline vs Adapter BOUN	26	71	19.959	<0.0001 ***
Baseline vs Morph V2	130	4	116.604	<0.0001 ***
Morph V2 vs Morph V3	20	2	13.136	0.0003 ***
Baseline vs A2b BiLSTM+POS	17	128	83.448	<0.0001 ***
A2b vs Adapter BOUN	100	34	31.530	<0.0001 ***
Adapter BOUN vs Morph V2	178	7	156.216	<0.0001 ***
A1a vs A1b — BiLSTM effect	15	102	63.214	<0.0001 ***
A2a vs A2b — POS feature effect	27	51	6.782	0.0092 **
<i>PUD zero-shot</i>				
Baseline vs Adapter BOUN	3	15	6.722	0.0095 **
Baseline vs Morph V2	27	0	25.037	<0.0001 ***
Morph V2 vs Morph V3	3	0	1.333	0.2482 n.s.

Table 3: McNemar’s continuity-corrected χ^2 tests (sentence-level LAS). b : sentences where system A is correct and B is not; c : the reverse. * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$.

All improvements over the baseline are highly significant ($p < 0.001$) on BOUN. On PUD zero-shot, the adapter gain over the baseline is significant ($p = 0.0095$) and the morphological injection gain is highly significant ($p < 0.0001$). The difference between V2 and V3 is not significant on PUD ($p = 0.2482$), confirming that Zemberek labels add no generalizable information beyond UD FEATS.

5.6 Per-Relation LAS

Table 4 in Appendix H shows per-relation LAS for the 25 most frequent dependency relations on BOUN. A2b achieves the highest LAS on the majority of relations. The largest gains over the baseline appear on `case` (+19.38), `obl` (+13.17), `acl` (+11.11), `nmod:poss` (+10.55), and `amod` (+13.31) — all relations that depend heavily on case marking and morphological agreement. The adapter provides consistent but smaller gains. Morph V2 and V3 per-relation scores are computed from `parser.predict()` output and match the training-loop headline LAS of 71.79.

6 Conclusion

We presented a modular Turkish dependency parsing system grounded in six course papers. Our ablation study, to the best of our knowledge, is the first application of the STEPS framework to Turkish. This shows that a BiLSTM on top of BERTurk is the single most impactful component, with gains concentrated in label accuracy rather than head selection, consistent with the importance of sequential context for Turkish morphological suffix disambiguation. POS input features add a smaller but significant further improvement. Together, the best ablation configuration (A2b: BiLSTM + BERTurk + POS) achieves 74.73 LAS on BOUN, outperforming all off-the-shelf parsers. Morphological feature injection from the UD FEATS column improves label prediction substantially over the baseline, while treebank-specific Houlsby adapters achieve the strongest zero-shot transfer to PUD (63.87 LAS), showing that the frozen BERTurk encoder generalizes well across domains.

Three directions remain open. First, combining A2b with UD FEATS injection is the most natural next step and is expected to push results closer to Özateş et al. (2022). Second, the adapter system underperformed A2b on BOUN, likely because a frozen encoder limits label prediction; a partially unfrozen variant warrants investigation. Third, closing the remaining gap to the hybrid system of Özateş et al. (2022) likely requires hand-written morphological rules, which lies beyond the scope of a purely data-driven approach.

7 Acknowledgements

Claude (Anthropic) was used for assisting in coding and debugging, especially for SuPar compatibility patches, custom Houlsby adapter implementation in PyTorch, morphological feature batch conversion, and PyTorch/Transformers version conflict resolution. All experimental design, implementation decisions, analyses, and reports are the authors’ own.

References

- Ahmet A. Akın and Mehmet D. Akın. 2007. Zemberek, an Open Source NLP Framework for Turkic Languages. In *Structure*.
- Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. 2020. Unsupervised Cross-lingual Representation Learning at Scale. In *Proceedings of ACL 2020*, pages 8440–8451.
- Timothy Dozat and Christopher D. Manning. 2017. Deep Biaffine Attention for Neural Dependency Parsing. In *Proceedings of ICLR 2017*.
- Stefan Grünewald, Annemarie Friedrich, and Jonas Kuhn. 2021. Applying Occam’s Razor to Transformer-Based Dependency Parsing: What Works, What Doesn’t, and What Is Really Necessary. In *Proceedings of IWPT 2021*, pages 131–144.
- Neil Houlsby, Andrei Giurgiu, Stanisław Jastrzębski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-Efficient Transfer Learning for NLP. In *Proceedings of ICML 2019*, pages 2790–2799.
- Quinn McNemar. 1947. Note on the Sampling Error of the Difference between Correlated Proportions or Percentages. *Psychometrika*, 12(2):153–157.
- Joakim Nivre, Marie-Catherine de Marneffe, Filip Ginter, Yoav Goldberg, Jan Hajič, Christopher D. Manning, Ryan McDonald, Slav Petrov, Sampo Pyysalo, Natalia Silveira, Reut Tsarfaty, and Daniel Zeman. 2016. Universal Dependencies v1: A Multilingual Treebank Collection. In *Proceedings of LREC 2016*, pages 1659–1666.
- Şaziye Betül Özateş, Arzucan Özgür, Tunga Güngör, and Balkız Öztürk. 2022. A Hybrid Deep Dependency Parsing Approach Enhanced with Rules and Morphology: A Case Study for Turkish. *IEEE Access*, 10:93867–93886.
- Peng Qi, Yuhao Zhang, Yuhui Zhang, Jason Bolton, and Christopher D. Manning. 2020. Stanza: A Python Natural Language Processing Toolkit for Many Human Languages. In *Proceedings of ACL 2020: System Demonstrations*, pages 101–108.
- Stefan Schweter. 2020. *BERTurk – BERT Models for Turkish*. Zenodo.
- Milan Straka and Jana Straková. 2017. Tokenizing, POS Tagging, Lemmatizing and Parsing UD 2.0 with UDPipe. In *Proceedings of the CoNLL 2017 Shared Task*, pages 88–99.
- Utku Türk, Furkan Atmaca, Şaziye Betül Özateş, Gözde Berk, Seyyit Talha Bedir, Abdullatif Köksal, Balkız Öztürk Başaran, Tunga Güngör, and Arzucan Özgür. 2022. Resources for Turkish Dependency Parsing: Introducing the BOUN Treebank and the BoAT Annotation Tool. *Language Resources & Evaluation*, 56(1):259–307.
- Ahmet Üstün, Arianna Bisazza, Gosse Bouma, and Gertjan van Noord. 2022. UDapter: Typology-Based Language Adapters for Multilingual Dependency Parsing and Sequence Labeling. *Computational Linguistics*, 48(3):555–592.
- Yu Zhang, Zhenghua Li, and Min Zhang. 2020. Efficient Second-Order TreeCRF for Neural Dependency Parsing. In *Proceedings of ACL 2020*, pages 3295–3305.
- Houquan Zhou and Sandra Kübler. 2025. Cross-Lingual Dependency Parsing for a Language with a Unique Script. *Natural Language Processing*, 31(2):277–305.

Appendices

This appendix lists every notebook used in the project, together with its purpose, key outputs, and shareable Google Drive URL. All notebooks were run on Google Colab Pro with an Nvidia T4 GPU.

Appendix A – 01 Setup – Environment and Data Setup.

- **Notebook:** 01_setup.ipynb
- **Purpose:** Installs required libraries, clones the three UD Turkish treebanks, verifies split counts, extracts the FEATS vocabulary, and saves feat_vocab.json.
- **URL:** <https://colab.research.google.com/drive/1RJJmSMZpMFK58J1IT4snb5vxOWNPVhD4>

Appendix B – 02 Baseline – Training and Evaluation.

- **Notebook:** 02_baseline_final.ipynb
- **Purpose:** Trains the CRF2oDependencyParser with BERTurk encoder separately on BOUN and IMST. Evaluates on the respective test sets and saves results.
- **URL:** https://colab.research.google.com/drive/1lACkaPLexCiLrNxPO01Ap1kqwchHgUq_

Appendix C – 03 Ablation – STEPS Study.

- **Notebook:** 03_ablation.ipynb
- **Purpose:** Runs the STEPS ablation on BOUN. Tests three binary comparisons: BiLSTM layer presence, POS input features, and encoder choice (BERTurk vs XLM-R).
- **URL:** https://colab.research.google.com/drive/1LfUZIopDYaTo_iI6KL3Q3XYDSiGxTmyh

Appendix D – 04 Adapters – Houlby Multi-Treebank Training.

- **Notebook:** 04_adapters.ipynb
- **Purpose:** Implements Houlby adapter modules from scratch in PyTorch and trains them jointly over BOUN and IMST using a round-robin schedule with a frozen BERTurk encoder.
- **URL:** <https://colab.research.google.com/drive/1y6EbJ5DrVzcJgSr2Lz4TxzkuTqM3Fhhh>

Appendix E – 05a Morph UD FEATS – Feature Injection.

- **Notebook:** 05a_morph_ud_feats.ipynb
- **Purpose:** Final implementation of V2 morphological feature injection. Rebuilds the feature vocabulary from all three treebanks and fine-tunes with the morphological embedding and projection modules.
- **URL:** https://colab.research.google.com/drive/1KoZGJF8ub1B4PSjmNDpG_PRe7r6rA0V7

Appendix F – 05b Morph Zemberek – Enriched Injection.

- **Notebook:** 05b_morph_zemberek.ipynb
- **Purpose:** Extends V2 with a second morphological embedding table built from Zemberek-NLP morpheme labels.
- **URL:** https://colab.research.google.com/drive/1unHJFrlr_Flrpma9i52dNfCaMcHyd9ie

Appendix G – 06 Final Comparison – Per-Relation Analysis.

- **Notebook:** 06_final_comparison.ipynb
- **Purpose:** Consolidates results across all phases and generates the per-relation LAS breakdown for the top-10 most frequent dependency relations.
- **URL:** https://colab.research.google.com/drive/1p-xHb1Fzgl0Do05Nwtp5_QD9rbVmBAPU

Appendix H — Per-Relation LAS Results

Relation	N	Base	A2b	Adap.	V2†	V3†
punct	2026	73.89	78.43	74.78	73.89	73.89
nmod:poss	1053	74.07	84.62	82.43	74.07	74.07
obl	987	62.11	75.28	65.96	62.11	62.11
root	979	84.88	88.97	83.25	84.88	84.88
nsubj	829	63.69	76.36	72.86	63.69	63.69
amod	766	58.49	71.80	60.97	58.49	58.49
obj	728	65.80	72.25	71.98	65.80	65.80
conj	679	48.60	63.62	55.96	48.60	48.60
det	524	86.45	91.98	89.89	86.45	86.45
advmod	436	60.09	70.87	65.83	60.09	60.09
nmod	366	12.30	43.44	34.15	12.30	12.30
acl	351	70.37	81.48	66.67	70.37	70.37
advcl	317	53.63	64.35	63.09	53.63	53.63
case	294	65.99	85.37	69.05	65.99	65.99
cc	258	74.81	72.87	64.73	74.81	74.81
compound	217	27.19	42.86	42.40	27.19	27.19
cop	192	77.08	83.85	77.60	77.08	77.08
flat	181	65.19	77.90	74.03	65.19	65.19
ccomp	173	61.27	82.08	71.68	61.27	61.27
nummod	163	50.92	74.85	71.78	50.92	50.92
advmod:emph	151	81.46	86.75	83.44	81.46	81.46
compound:lvc	118	77.12	83.05	81.36	77.12	77.12
csubj	72	0.00	61.11	58.33	0.00	0.00
discourse	52	0.00	38.46	26.92	0.00	0.00
compound:redup	50	0.00	66.00	54.00	0.00	0.00

Table 4: Per-relation LAS on BOUN test (top 25 by frequency). † Morph V2/V3 per-relation computed from `parser.predict()` output; headline LAS (V2: 71.79, V3: 71.79) comes from training-loop evaluation.

Notable patterns. `csubj`, `discourse`, and `compound:redup` score 0.00 for the baseline and morph variants but are substantially improved by A2b (+61.11, +38.46, +66.00 respectively), reflecting that the BiLSTM and POS features are critical for rare and structurally complex relations. The adapter also improves these, though to a lesser extent. The morph injection variants show no improvement on these relations over the baseline, suggesting that UD FEATS alone are insufficient for low-frequency, structurally complex dependencies. We note that V2 and V3 per-relation scores are computed from `parser.predict()` rather than the training-loop evaluation path; since morph injection is applied via a monkey-patched forward pass, the `predict()` path may not fully activate the injection, which likely explains why per-relation scores for V2 and V3 match the baseline rather than reflecting the headline LAS gain of 9.83.

Appendix I — Progress Report Baseline Results

Table 5 reports the baseline results from our progress report submission, where all systems were trained for 10 epochs with a shared learning rate schedule. These numbers differ from the final results in Table 1 for two reasons. First, the baseline was retrained with 30 epochs and patience 10, and per-treebank learning rates were tuned empirically (BOUN: 1×10^{-3} , IMST: 2×10^{-3}). Second, the XLM-R variant (A3b) was retrained with a corrected lower learning rate. The progress report adapter results are also included for reference; these were produced with 10 training epochs before the final 30-epoch rerun.

System	Test	UAS	LAS
<i>Baseline (10 epochs)</i>			
BERTurk + SuPar (BOUN train)	BOUN	73.41	43.87
BERTurk + SuPar (BOUN train)	PUD	72.76	44.17
BERTurk + SuPar (IMST train)	IMST	69.68	14.95
BERTurk + SuPar (IMST train)	PUD	68.35	53.62
<i>Ablation on BOUN (10 epochs)</i>			
A1a: BERTurk, no BiLSTM	BOUN	73.41	43.87
A1b: BiLSTM + BERTurk	BOUN	79.25	67.80
A2b: BiLSTM + BERTurk + POS	BOUN	80.40	71.26
A2b: BiLSTM + BERTurk + POS	PUD	80.47	65.52
A3b: XLM-R (original LR)	BOUN	25.12	0.32
<i>Adapters (10 epochs)</i>			
Adapter (BOUN)	BOUN	82.19	74.77
Adapter (IMST)	IMST	82.47	74.51
Adapter (BOUN, zero-shot)	PUD	78.66	67.69
Adapter (IMST, zero-shot)	PUD	74.15	64.22

Table 5: Baseline and ablation results from the progress report (10-epoch runs). The IMST baseline LAS (14.95) reflects label-head non-convergence due to an aggressive learning rate decay on the smaller IMST dataset, which was corrected in the final system. The XLM-R result (0.32 LAS) reflects an unsuitably high learning rate, corrected in the final rerun (49.39 LAS).